

Big Data Pipeline for Diabetes Analysis using Apache Spark and MongoDB

Final Project Report

Group 11

Team Member:

- Prayusha Poudel
- Tuo Yan
- Dan Le

1. Introduction

Dataset and problem statement

This project analyzes a diabetes healthcare dataset containing 100,000 patient records with features such as age, gender, BMI, blood glucose level, HbA1c, hypertension, heart disease, smoking history, location, and diabetes status. The goal is to build an end-to-end pipeline that can ingest the data into MongoDB, process it with Apache Spark, and generate useful insights about diabetes prevalence and risk factors.

Big Data relevance

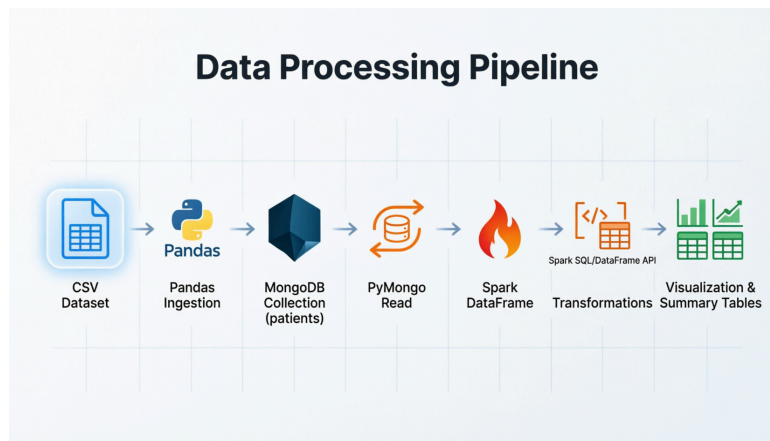
Healthcare data is high-volume, heterogeneous, and computationally expensive to analyze with traditional single-machine tools. Spark provides distributed processing for cleaning, aggregation, SQL analysis, and ranking tasks, while MongoDB supports flexible JSON-like documents that are suitable for semi-structured health records and fast operational querying.

2. Methodology

Dataset details and preprocessing

- Dataset size: 100,000 records.
- Main fields used in analysis: age, gender, bmi, blood_glucose_level, hbA1c_level, hypertension, heart_disease, smoking_history, location, and diabetes.
- Preprocessing in Spark: remove records with null age or BMI, cast age to integer, cast bmi and blood_glucose_level to double, and cast diabetes to integer.
- The cleaned DataFrame is cached to speed up repeated analytical queries.

Architecture of the Spark + MongoDB pipeline



Architecture of the Solution

The system follows a layered Big Data architecture consisting of four main components: data ingestion, storage, processing, and visualization.

- **Data Ingestion Layer:**
The raw diabetes dataset (CSV format) is first loaded into the system using Python and Pandas. This step ensures compatibility with MongoDB by converting the data into JSON-like documents.
- **Storage Layer (MongoDB):**
MongoDB is used as a NoSQL database to store patient records in a flexible document format. Each record represents a patient, enabling efficient retrieval without joins. This layer is optimized for operational queries and scalable storage.
- **Processing Layer (Apache Spark):**
Apache Spark is used for distributed data processing. The data is loaded from MongoDB into Spark DataFrames, where transformations, aggregations, and SQL queries are executed in parallel across partitions. Spark's lazy evaluation model ensures that computations are optimized before execution.
- **Output & Visualization Layer:**
Aggregated results are converted into Pandas DataFrames for visualization using charts and summary tables. This layer provides insights into diabetes trends and risk factors.

This architecture separates storage and computation, allowing MongoDB to handle data persistence while Spark performs scalable analytics, which is essential in Big Data systems.

Schema design

MongoDB collection structure

Each patient is stored as one document in the **patients** collection. A document-centric design is appropriate because each row already represents a single patient entity and the attributes vary in type (numeric, categorical, and binary indicators). This avoids joins during operational reads and makes the schema easy to evolve if additional medical attributes are added later.

- Numeric analytical fields: age, bmi, blood_glucose_level, hbA1c_level.
- Categorical fields: gender, smoking_history, location.
- Binary/indicator fields: diabetes, hypertension, heart_disease, and race indicator columns.

Indexing strategy

Implemented index

The notebook creates an ascending index on **location** because several analyses group or filter by location when comparing diabetes prevalence across regions.

- Implemented: `collection.create_index([('location', pymongo.ASCENDING)])`.
- Rationale: improves lookup speed for location-based queries and supports the location aggregation shown in the report.
- Further useful indexes for future work: compound indexes such as (location, diabetes) or single-field indexes on `blood_glucose_level` and `diabetes` for common filtering patterns.

Conceptual sharding strategy

Although sharding was not deployed in the local notebook environment, a practical production strategy would be to shard the patients collection by location or by a hashed patient identifier. Location-based sharding would keep geographically grouped queries efficient, while hashed sharding would distribute writes more evenly across shards. For this project, a balanced recommendation is to use a hashed shard key for write scalability and keep a supporting index on location for regional analytics.

Spark partitioning strategy and justification

Current processing

Spark already splits the DataFrame into partitions for parallel execution. The notebook also caches the cleaned DataFrame because multiple actions and queries reuse the same transformed dataset.

Justification

The workload contains repeated aggregations by location, age group, and diabetes status, plus a window function partitioned by gender. Partitioned execution lets Spark process these groups concurrently instead of scanning the full dataset serially.

Recommended optimization

For larger deployments, repartitioning by a high-usage analytical field such as location before wide aggregations can reduce shuffle imbalance. For example, repartitioning by location would align with the location-based diabetes-rate query, while the gender-based window function already uses logical partitioning through `Window.partitionBy('gender')`.

3. Implementation

Pipeline Steps

1. Initialize Spark session and MongoDB connection
2. Load CSV dataset and insert into MongoDB
3. Define schema, indexing, and sharding strategy
4. Load data into Spark DataFrame

5. Remove `_id` and clean data
6. Cast columns to correct types
7. Cache cleaned DataFrame
8. Register Spark SQL temporary view
9. Execute aggregation queries
10. Apply window functions for ranking
11. Benchmark MongoDB query performance
12. Convert results to Pandas
13. Generate visualizations
14. Store selected results back to MongoDB

Spark transformations and queries performed

- Filtering: remove records with null age and BMI.
- Type conversion: cast age, bmi, blood_glucose_level, and diabetes to analytical types.
- Aggregation with Spark SQL: count total patients, average BMI, and diabetes percentage by location.
- Age-band query with Spark SQL: Under 30, 30–49, 50–64, and 65+ groups.
- DataFrame aggregation: `groupBy('diabetes')` to compute patient count, average age, average BMI, and average glucose.
- Window function: rank patients by blood_glucose_level within each gender using `Window.partitionBy('gender').orderBy(desc('blood_glucose_level'))`.

MongoDB integration and performance tuning

MongoDB integration

MongoDB is used as the operational storage layer, while Spark handles heavy analytical processing. PyMongo is used for insertion and direct query benchmarking, and the MongoDB Spark connector is used to move the dataset into Spark.

Performance tuning

The notebook benchmarks the same query on the **location='Alabama'** filter before and after creating the index. Caching the cleaned Spark DataFrame also reduces repeated recomputation during analysis.

4. Results & Insights

Key findings from the analysis

- Clean dataset size after preprocessing: 100,000 records.
- Diabetic patients: 8,500 (8.5%); non-diabetic patients: 91,500.
- Average profile by diabetes status: diabetic patients have higher average age (60.95 vs. 40.10), higher BMI (31.99 vs. 26.89), and higher blood glucose (194.09 vs. 132.85).
- Age-group analysis shows diabetes prevalence rises sharply with age: 0.84% for Under 30, 5.19% for ages 30–49, 14.02% for ages 50–64, and 20.45% for ages 65+.
- Location analysis identifies states such as Delaware, Kansas, Illinois, and Montana among the highest diabetes-percentage groups in the notebook output.

- The gender-partitioned ranking query identifies the highest blood-glucose patients within each gender group for risk-oriented review.

Performance evaluation with exact benchmark numbers

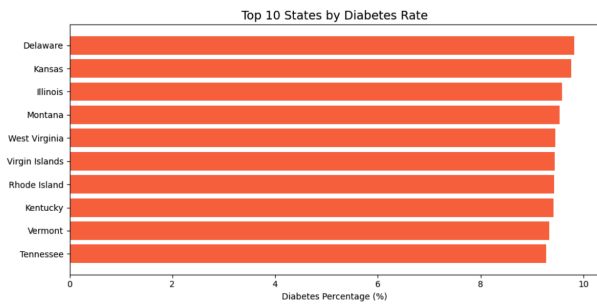
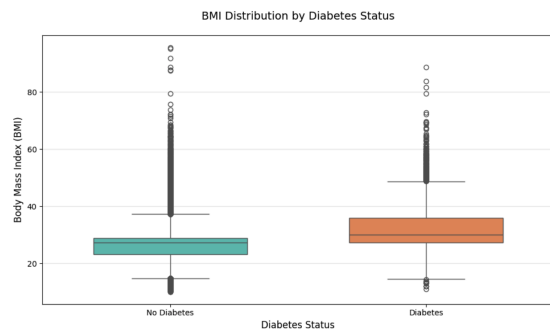
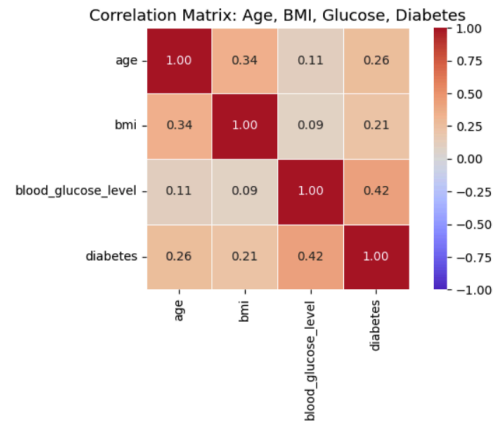
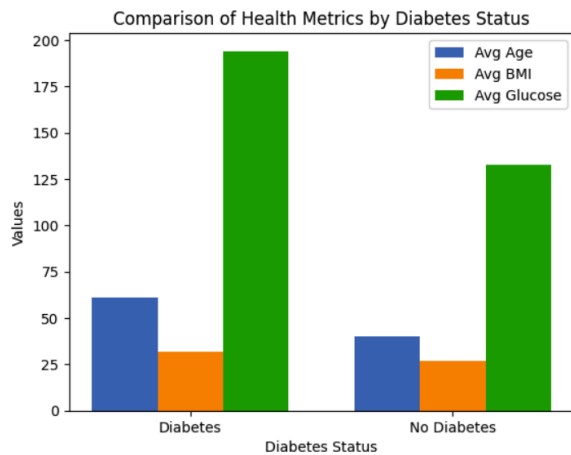
Scenario	Filter	Rows Returned	Execution Time
Without index	location = 'Alabama'	2036	0.0221 s
With index	location = 'Alabama'	2036	0.0203 s

Measured speedup

The indexed query completed in 0.0203 seconds compared with 0.0221 seconds without the index, which is approximately 1.1× faster in this local experiment. The modest gain is expected because the dataset is relatively small (100k records) and the benchmark was run on a single local environment rather than a distributed production cluster.

Visualizations and interpretation

- **Histogram:** used to compare age distribution by diabetes status and show that older populations tend to have higher diabetes prevalence.
- **Grouped bar chart:** used to compare average age, BMI, and blood glucose levels between diabetic and non-diabetic patients.
- **Boxplot:** used to visualize BMI distribution and outliers across diabetes groups.
- **Heatmap (correlation matrix):** used to summarize relationships among age, BMI, blood glucose level, and diabetes.
- **Horizontal bar chart:** used to present the top 10 states with the highest diabetes rates.
- **Summary tables:** used to present grouped counts and average metrics clearly for reporting.



5. Conclusion & Future Work Challenges

The main challenges were configuring the Spark–MongoDB environment, handling schema cleanup for Spark processing, and validating query logic across both systems.

Conclusion

The project successfully demonstrates an end-to-end Big Data workflow that combines MongoDB storage with Spark analytics. Even on a local setup, the pipeline shows how indexing, distributed transformations, and structured analysis can support healthcare insight generation.

Future improvements

Next steps include deploying a real sharded MongoDB cluster, using explicit Spark repartitioning experiments, benchmarking at larger scale, adding machine learning models for diabetes prediction, and deploying the solution in a cloud environment.

6. Code Submission

[GitHub repository](#)